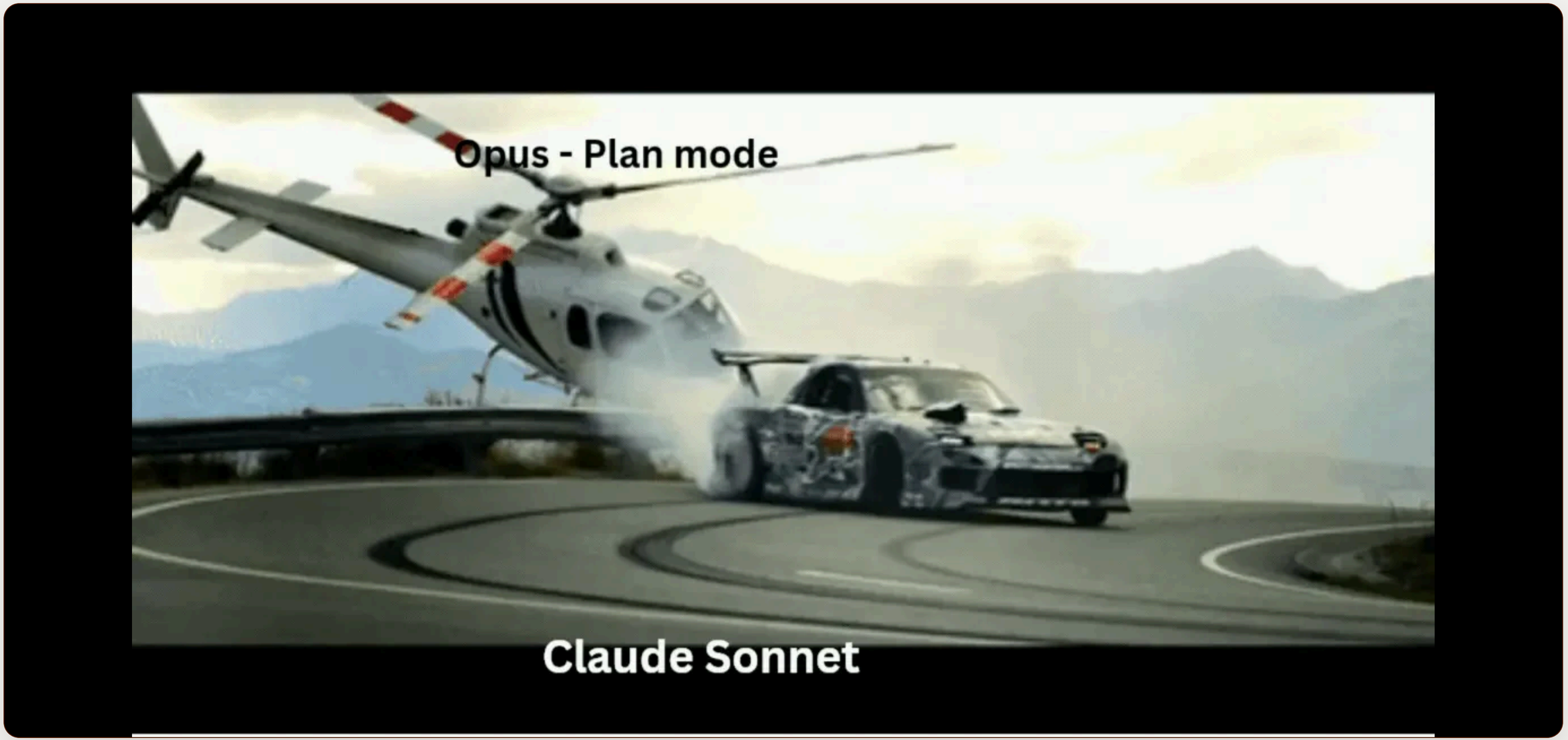
# Paradigma Cloud: API Token & Fuerza Bruta



# Paradigma Local: Tracción, Control & Coste Cero

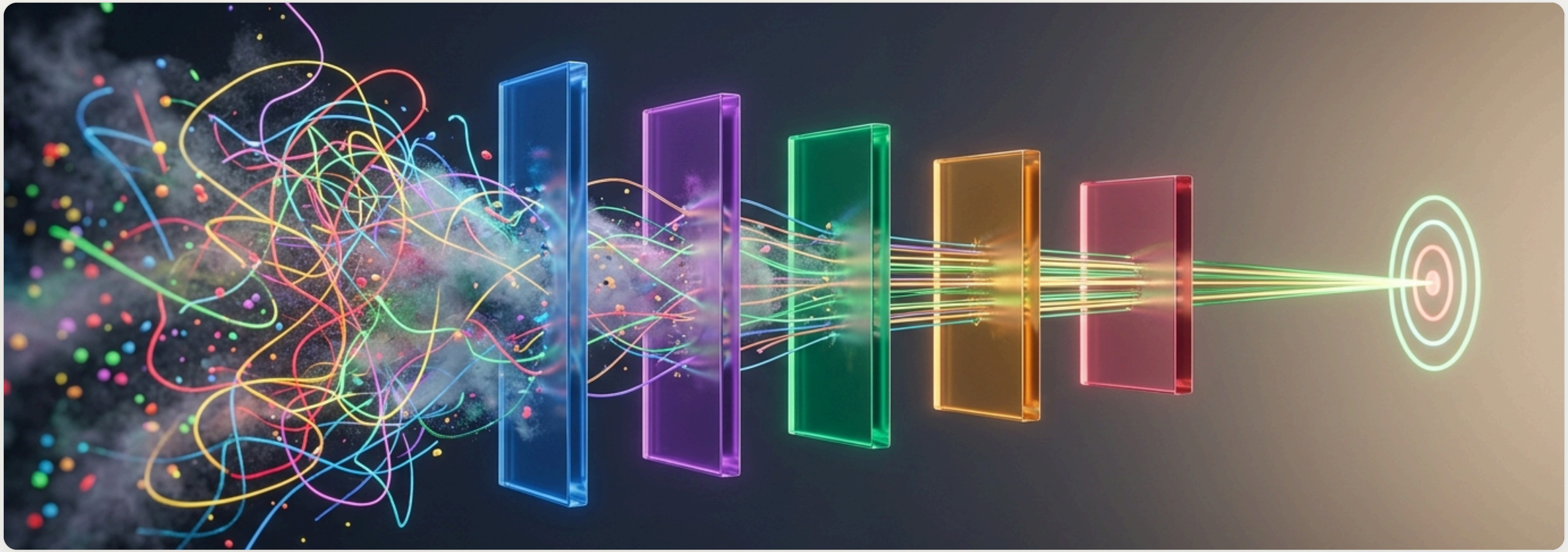


“

**Cuando te equivocas**,
con una **API**
tu enemigo es el **dinero**.
Con **IA local**,
tu enemigo es el **tiempo**.

# El Embudo del SDLC: Del Caos a la Certeza

Cada fase del SDLC es un filtro que reduce la entropía hasta lograr una solución determinista y verificada.



🌪️ ENTRADA **Prompting Libre**

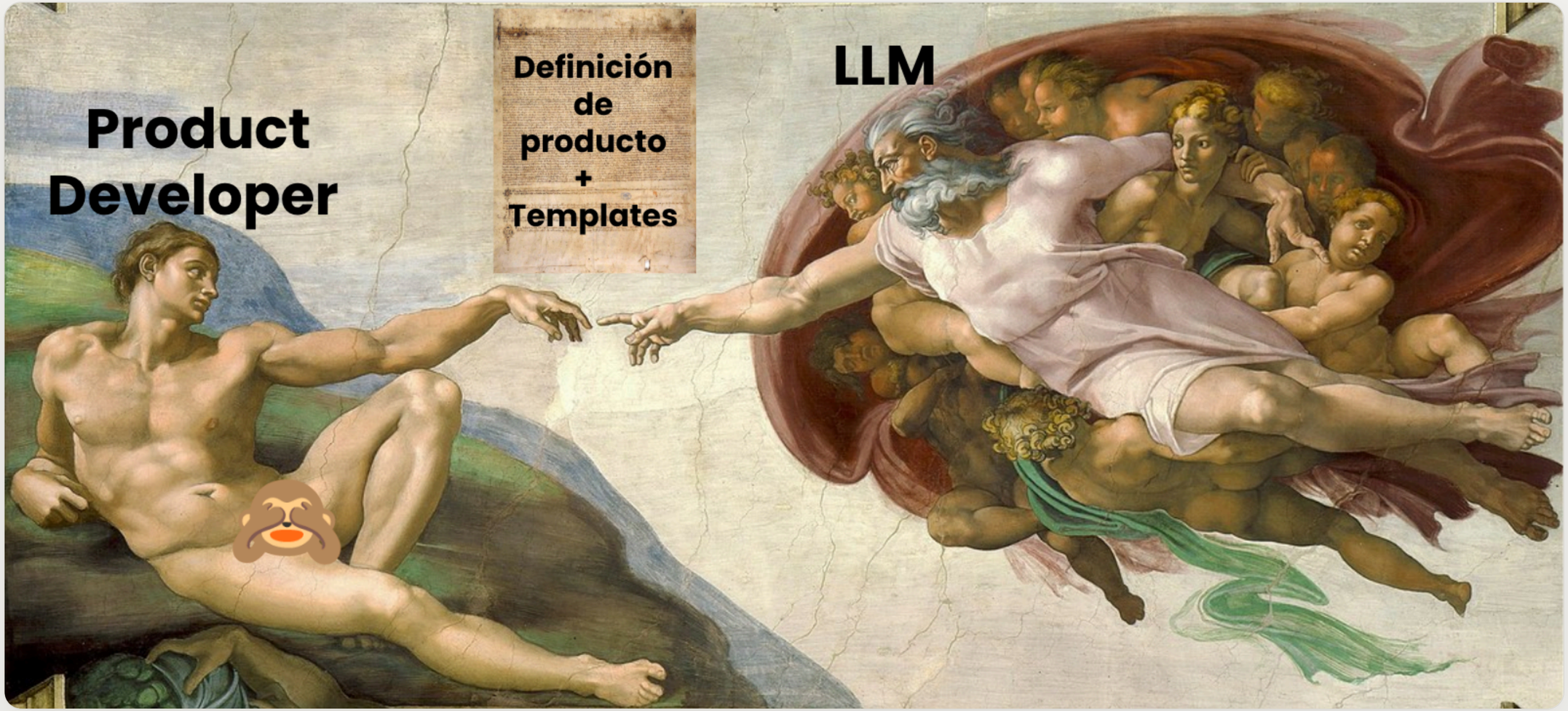
~$10^6$ caminos posibles • Alucinación y alcance infinito.

🎯 SALIDA **Certeza Verificada**

1 único camino verificado • Tests en verde y cero drift.

FASE 2 • EL ESLABÓN DEL PRODUCT DEVELOPER

# El Product Developer: ¿qué hay entre él y la IA?



# Los Tres Pilares: Código de Calidad con IA




PILAR 1 • PRODUCTO

### Inception & Story Mapping

**Decide:** Problema real, fronteras defensivas y valor MVP.

PILAR 2 • TECNOLOGÍA

### ADRs & Arquitectura

**Decide:** Stack, límites arquitectónicos y ergonomía para IA.

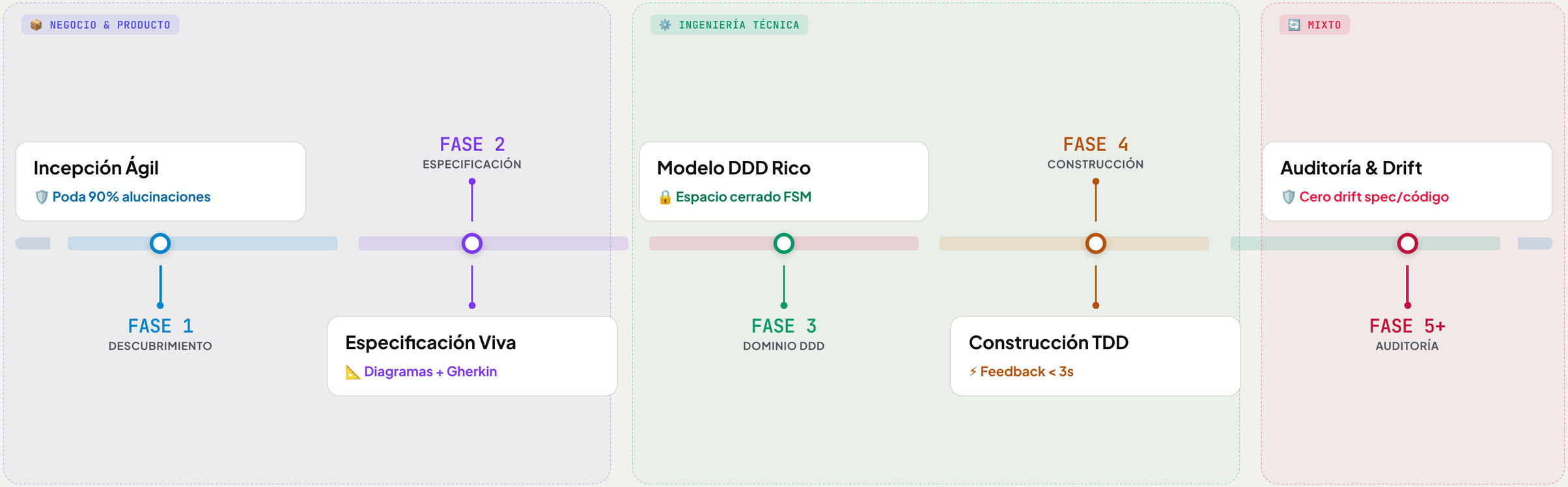
PILAR 3 • VERIFICACIÓN

### Tests & Tests de Arquitectura

**Decide:** Invariantes ejecutables, TDD y reglas arquitectónicas (ArchUnit / fitness functions).

# El Ciclo de Desarrollo con IA: 5 Fases Disciplinadas

Un pipeline estructurado donde cada fase acota el contexto mediante Skills especializadas.




💡 **Sinergia Crafter:** Producto acota el contrato (Fases 1–2), Ingeniería verifica con TDD (Fases 3–4) y ambos evolucionan sin drift (Fase 5+).

FASE 2 • ESPECIFICACIÓN VIVA • LO QUE FIJA EL DOCUMENTO

# ¿De qué hablan estos documentos? (¿memoria viva?)

usuarios

necesidades

*objetivos*

flujo de información

reglas de negocio

comportamiento

decisiones cerradas

invariantes

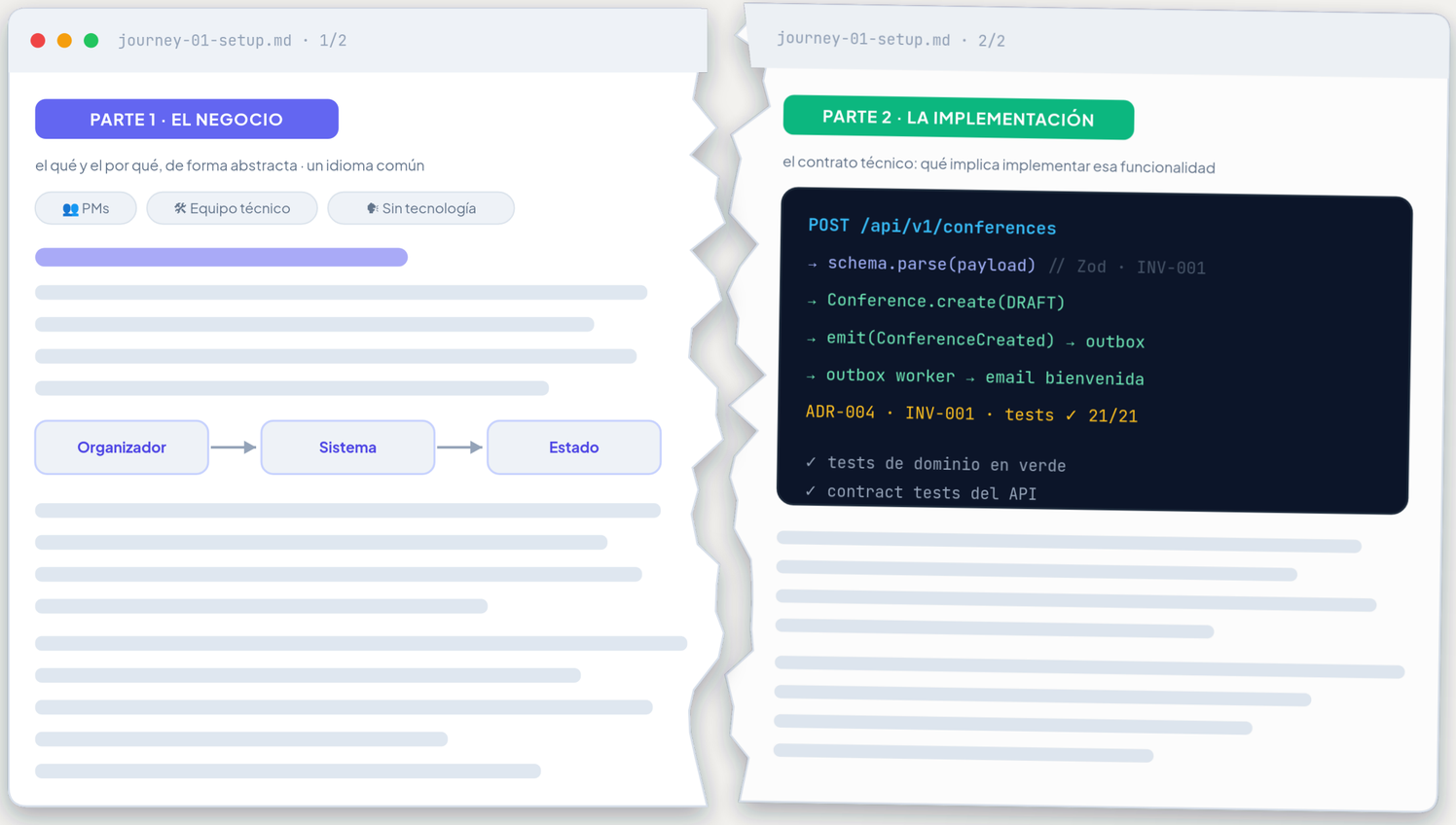
estados y transiciones

eventos asíncronos

📌 **Todo esto es lo que debería estar en nuestro día a día:** escrito donde el modelo lo entiende — y así nos comunicamos con él de forma efectiva.

FASE 2 • ESPECIFICACIÓN VIVA • FORMATO DE DOCUMENTO

# Un Documento, Dos Mitades: Negocio y Técnica



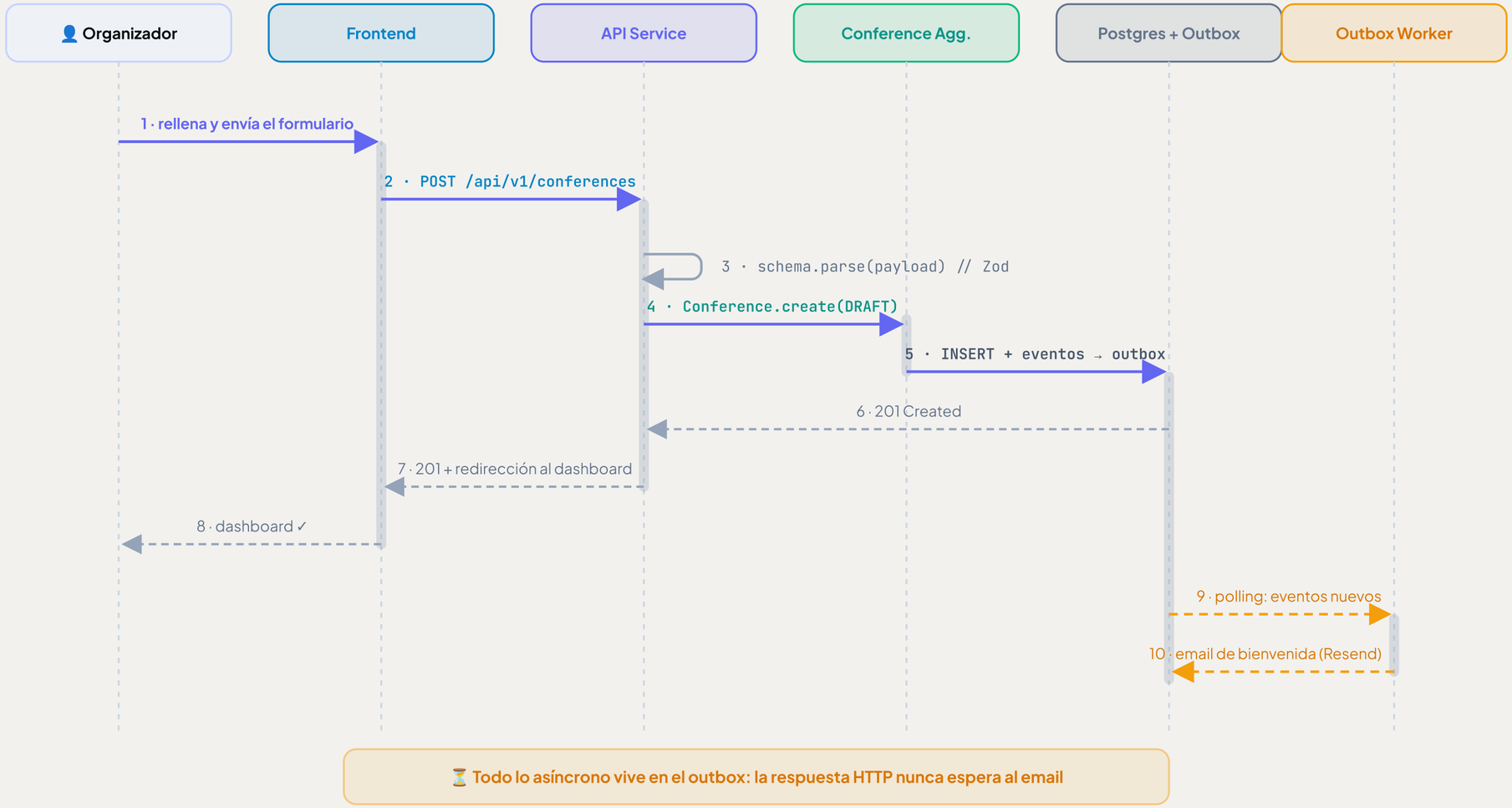

📖 **Leído por humanos**
Se entiende en menos de 2 minutos

⚡ **Pocos tokens**
Compacto para la IA: no infla el contexto

👁 **Flujos en visual**
Los esquemas explican el camino

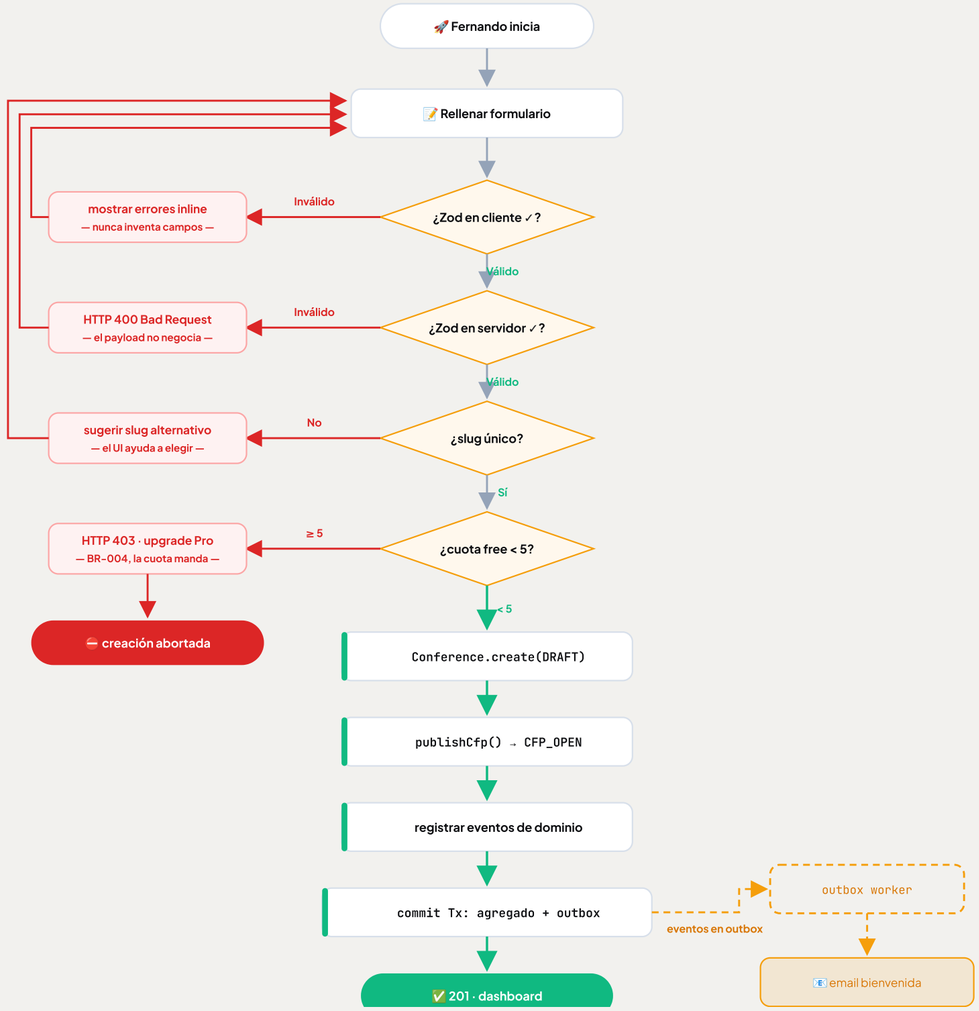
FASE 2 • ESPECIFICACIÓN VIVA • DIAGRAMA DE SECUENCIA

# Esquema 1 · La Secuencia: Quién habla con Quién



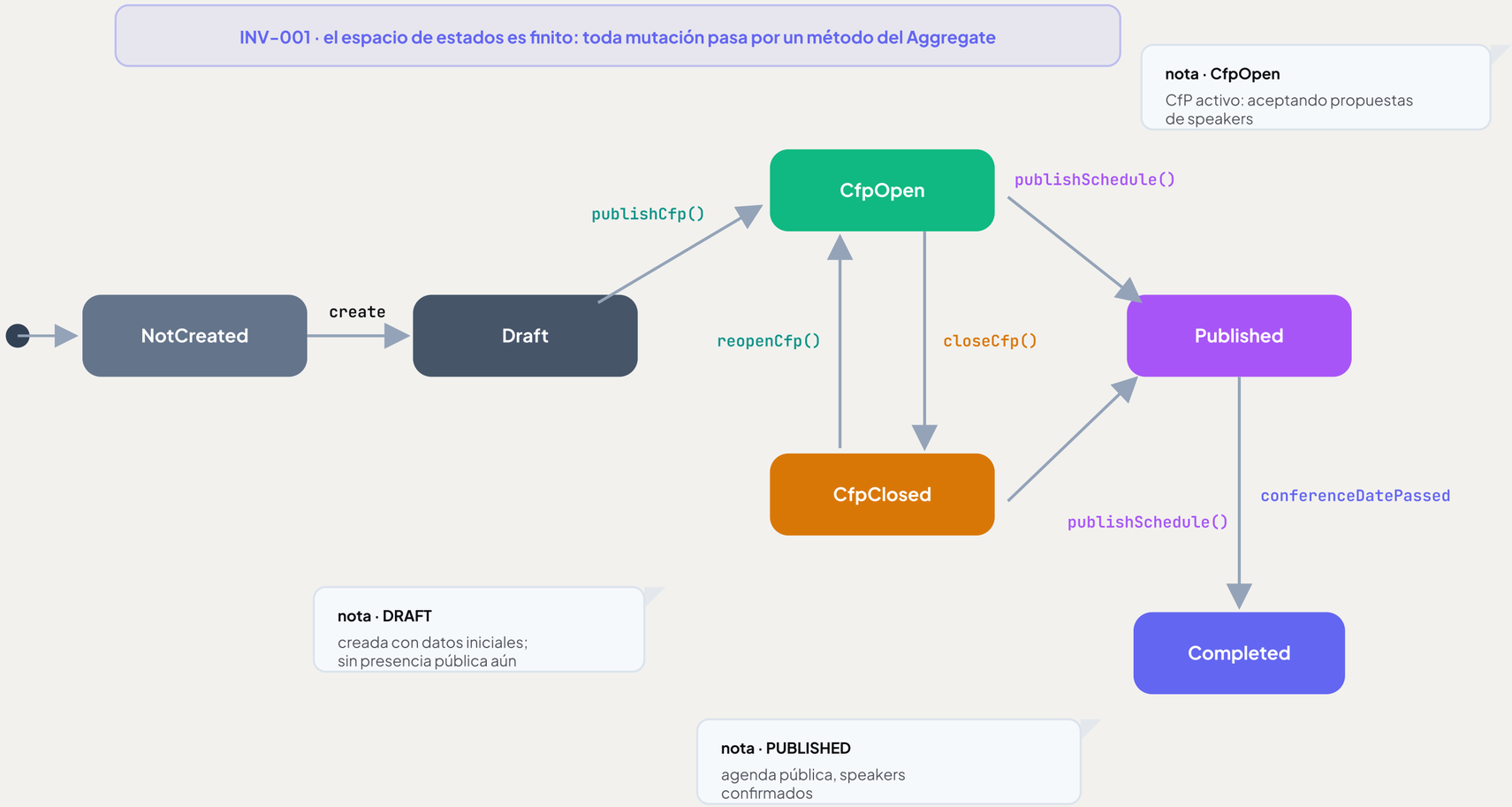
FASE 2 • ESPECIFICACIÓN VIVA • FLOWCHART DE DECISIONES

# Esquema 2 · El Flowchart: Toda Bifurcación, Cerrada



FASE 2 • ESPECIFICACIÓN VIVA • MÁQUINA DE ESTADOS (FSM)

# Esquema 3 · La FSM: Estados que no se Inventan



FASE 2 • ESPECIFICACIÓN VIVA • ESCRIBIR… ¿O MEJOR HABLAR?

# Hello, computer: escribir ya está anticuado

1986 → HOY

🖖

## Hello, computer

En 1986, a Scotty le dijeron que usara el teclado. Hoy el ordenador escucha de verdad — y con los esquemas y templates ya sabes exactamente qué pedirle.

📜 **Primero, el esquema**

Los gráficos y las plantillas fijan lo que quieres antes de abrir la boca.

✍️ **Luego, la voz**

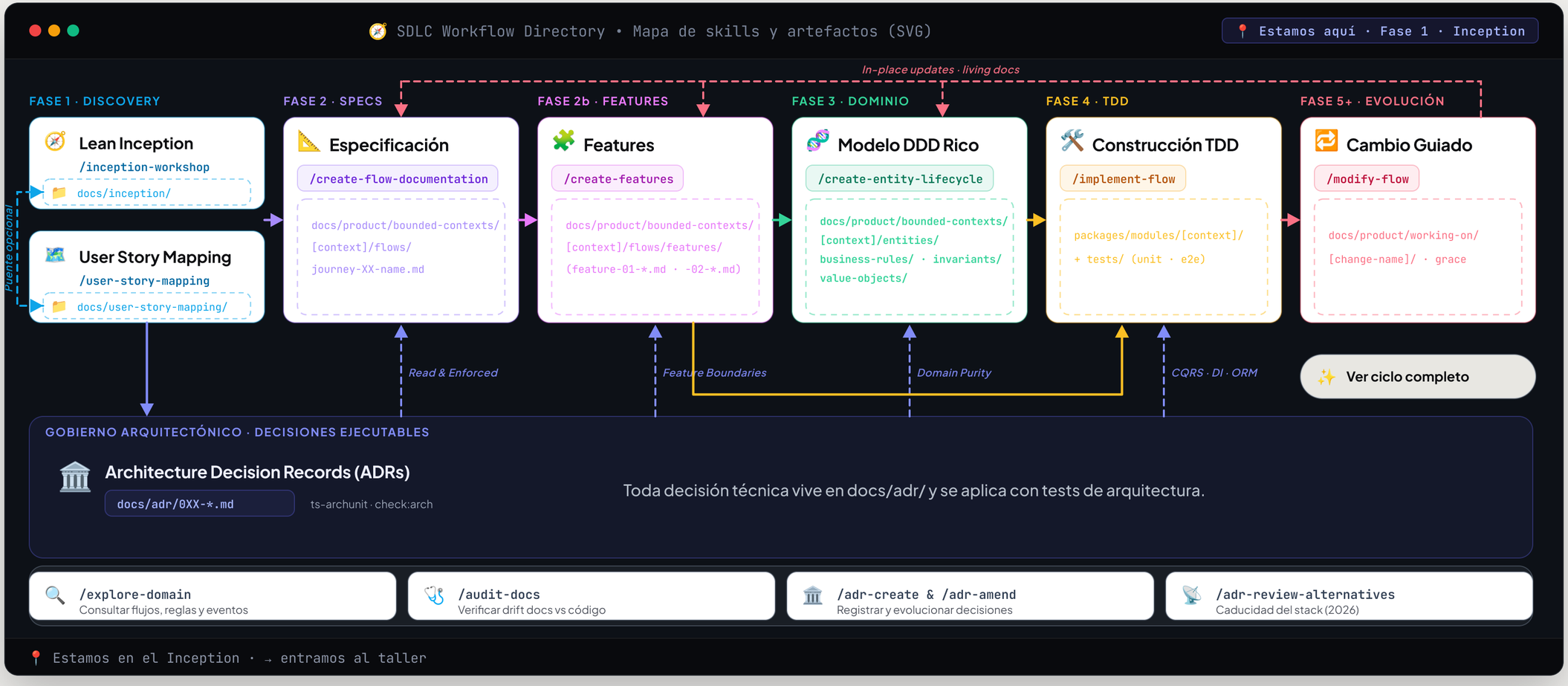
Hablar es más rápido que teclear: ya se me hace raro hacerlo de otro modo.

Habla con el ordenador

MAPA INTERACTIVO • DOCS/SDLC-WORKFLOW-DIRECTORY.MD

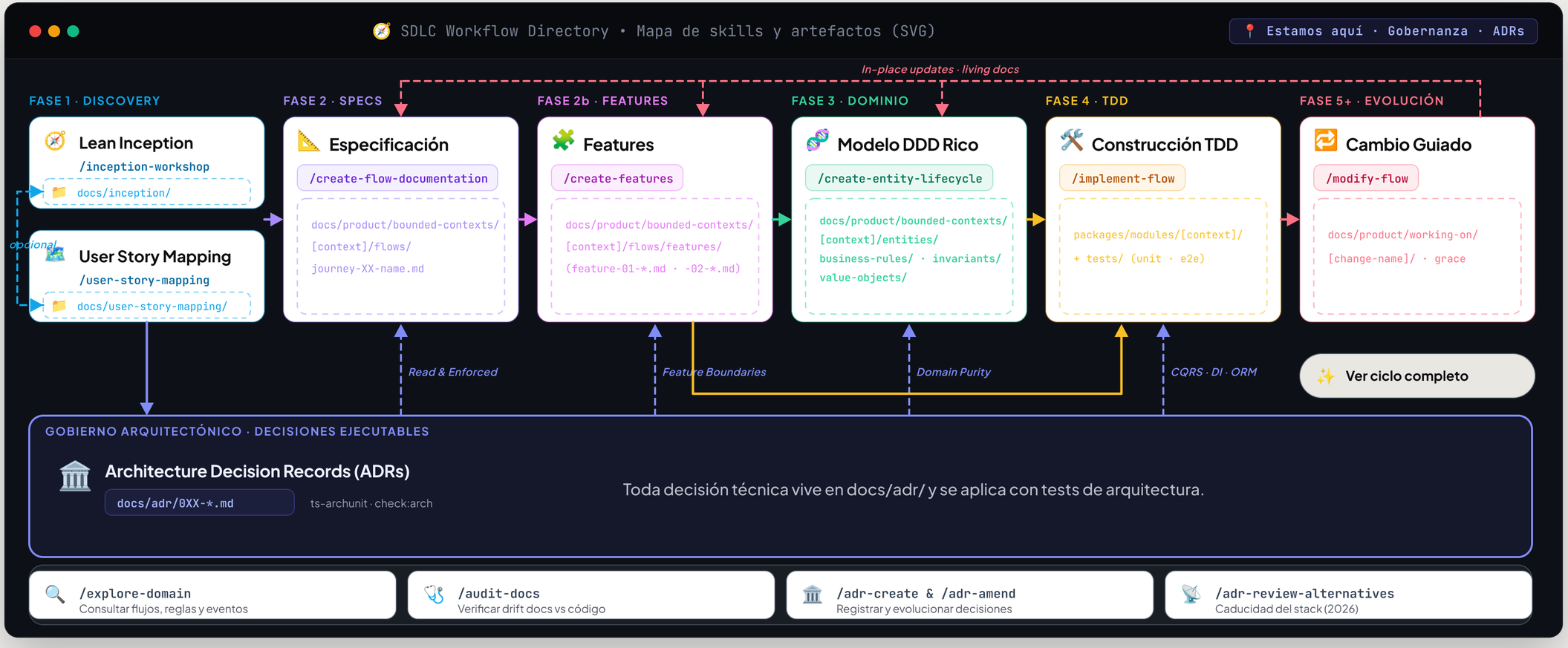
# El Mapa Real del SDLC: Skills, Artefactos y Gobierno

Skills encadenadas por artefactos; los ADRs gobiernan todo el ciclo.



## 📍 ESTAMOS AQUÍ • GOBERNANZA: ADRS

El dominio está blindado. ¿Quién garantiza que se respeta? La banda ADR: decisiones que el código no puede ignorar.



PILAR 2 • AI & AGENTIC ERGONOMICS (AX) • CORPUS ALIGNMENT VS. INERCIA DEL LLM

# La Fricción: ADRs vs. Pre-entrenamiento del Modelo




1. INERCIA DEL LLM

## El Sesgo de Internet

El modelo ha visto millones de repositorios mediocres y tutoriales obsoletos. Por defecto tenderá a **regresión a la media**: modelos anémicos y patrones genéricos.

2. GUARDRAIL TAX

## El Coste de Contención

Cuanto más choque tu ADR con los hábitos del modelo (ej. Controller Factories vs. NestJS), más **contexto, tipado defensivo y prompts** tendrás que pagar para mantenerlo en los raíles.

3. SOLUCIÓN CRAFTER

## Tipado & ts-archunit

No confíes en convencer al modelo con prosa: **blinda tus ADRs con contratos TypeScript estrictos y tests automáticos en CI** que rechacen cualquier alucinación en el acto.

EVIDENCIA TÉCNICA • VERCEL / NEXT.JS 16.3+ • DOCUMENTACIÓN & REPOSITORIOS

# Guardrail Tax: "This is NOT the Next.js you know"

NEXT.JS DOCS / GUIDES Next.js 16.3+

DOCUMENTACIÓN OFICIAL

## «Next.js empaqueta su documentación en node_modules para evitar el drift de los agentes»

A partir de Next.js 16.3+, Vercel incluye especificaciones completas dentro de `node_modules/next/dist/docs/`. Así los agentes consultan la versión exacta instalada sin peticiones externas y sin depender de datos de entrenamiento desfasados.

🔗 nextjs.org/docs/app/guides/ai-agents ↗

GITHUB / VERCEL/NEXT.JS Discussion #92197

EVOLUCIÓN DEL TEXTO

## «De "ALWAYS read docs" a "This is NOT the Next.js you know": sincronización en upgrades»

La discusión técnica #92197 documenta la transición hacia el aviso explícito de incompatibilidad. Al actualizar dependencias con npm las reglas podían desfasarse, lo que motivó a integrar la actualización de `AGENTS.md` en `next upgrade`.

🔗 github.com/vercel/next.js/discussions/92197 ↗

VERCEL LABS / CANARY Monorepo & Labs

IMPLEMENTACIÓN TÉCNICA

## «Detección de agentes y bloques delimitados: generate-agent-files.js en next dev»

Al arrancar `next dev`, el entorno detecta si hay un agente activo e inyecta la sección delimitada `<!-- BEGIN:nextjs-agent-rules -->`. Esto respeta las reglas custom del desarrollador y permite exclusión limpia con `agentRules: false`.

🔗 github.com/vercel/next.js (canary) ↗

AGENT SKILLS / MBLODE (GITHUB) Arquitectura IA

ANÁLISIS DE ECOSISTEMA

## «Scaffold Next.js: la documentación local como estándar de verdad frente a la inercia del modelo»

El análisis de `scaffold-nextjs` confirma que los LLMs arrastran patrones obsoletos si no se acota su contexto. Que un framework deba gobernar activamente a los agentes refuerza la necesidad de ADRs y especificaciones deterministas en el repositorio.

🔗 blode.co/agent-skills/scaffold-nextjs ↗

ARTEFACTO REAL • AGENTS.MD • PARADA ABRUPTA Y GOBERNANZA DETERMINISTA

# El Artefacto Inyectado: "This is NOT the Next.js you know"

📄 **AGENTS.md** MANAGED BLOCK — `node_modules/next/dist/server/lib/generate-agent-files.js`

*<!— BEGIN:nextjs-agent-rules —>*

## This is NOT the Next.js you know

This version has breaking changes — APIs, conventions, and file structure may all differ from your training data. Read the relevant guide in `node_modules/next/dist/docs/` (resolved from this file's directory; in monorepos the `next` package may not be visible from the repo root) before writing any code. Heed deprecation notices.

This block is written and re-added by `next dev` — verify at `node_modules/next/dist/server/lib/generate-agent-files.js`. Removing it from a diff only re-creates the uncommitted change; committing it with your work keeps the tree clean.

*<!— END:nextjs-agent-rules —>*

Inyectado y reescrito deterministamente en next dev — github.com/vercel/next.js (canary) ↗

### ⚠️ El fallo de las advertencias suaves

Cuando las directrices usan frases amables o condicionales (*"Please check the docs"*), los LLMs tienden a tratarlas como sugerencias de baja prioridad frente a los millones de ejemplos de versiones anteriores que dominan sus pesos de entrenamiento.

### 🛑 Técnica de *pattern interruption*

El encabezado `# This is NOT the Next.js you know` funciona como una instrucción de parada abrupta. Al contradecir categóricamente la premisa previa del modelo, eleva la probabilidad de que ejecute la llamada a herramienta (*tool use* en `node_modules/next/dist/docs/`) antes de alucinar código.

### 🔄 El problema de Next.js en particular

Pocos frameworks acumularon tantas capas divergentes en tan poco tiempo: Pages Router (2021), App Router con fetch cacheado (2023), y Server Actions/Components asíncronos en 15 y 16. La única forma de evitar que mezcle 2021 con 2026 era forzarla a ignorar su memoria base y consultar la versión empaquetada.

PILAR 1 • DOCS/ADR/ADR-WORKFLOW.MD • FLUJO SECUENCIAL DE IZQUIERDA A DERECHA

# El Ciclo Canónico de un ADR de la Duda a la Ley

01 PROPOSED

🎯

## Dilema & Drivers

`/adr-create`

Nace de una encrucijada técnica real o cambio de stack. El skill **reserva el siguiente ID secuencial (0XX)** y documenta las fuerzas impulsoras.

- **Artefacto:** `docs/adr/0XX-*.md`
- **Foco:** Define drivers y problema real

🎯 Acota el problema

02 2-3 OPCIONES

⚖️

## Exploración Real

`Matriz Pros / Cons`

Prohibido el sesgo de confirmación. Se evalúan **2 o 3 alternativas viables** contrastando pros, contras, mantenimiento y deuda técnica.

- **Matriz:** Mínimo 2–3 opciones viables
- **Criterio:** Pros, contras y trade-offs

⚖️ Criterio sin sesgos

03 OBLIGATORIO

🤖

## Ergonomía IA (AX)

`Corpus & Guardrails`

¿Choca con el sesgo del LLM? Se evalúa el **Corpus Alignment**, el **Guardrail Tax** (coste en prompts y tests) y el riesgo de deriva.

- **Corpus Bias:** Hábitos previos del LLM
- **Guardrail Tax:** Coste en prompts y tests

🤖 Mitiga la inercia LLM

04 COMPROMISO

📌

## Decisión & Reglas

`Contrato Firme`

Elección formal con honestidad intelectual: se explicitan los **beneficios**, los **trade-offs asumidos** y qué queda terminantemente prohibido.

- **Contrato:** Elección formal e inmutable
- **Invariante:** Qué queda prohibido

📌 Reglas innegociables

05 ACCEPTED ✅

🛡️

## Auditoría & Sello

`/adr-validate`

Pasa la **rúbrica automática (Score ≥ Medium)** y se indexa en `docs/adr/README.md`. Queda blindado como constitución inmutable del código.

- **Rúbrica:** Score ≥ Medium para aceptar
- **Catálogo:** `docs/adr/README.md`

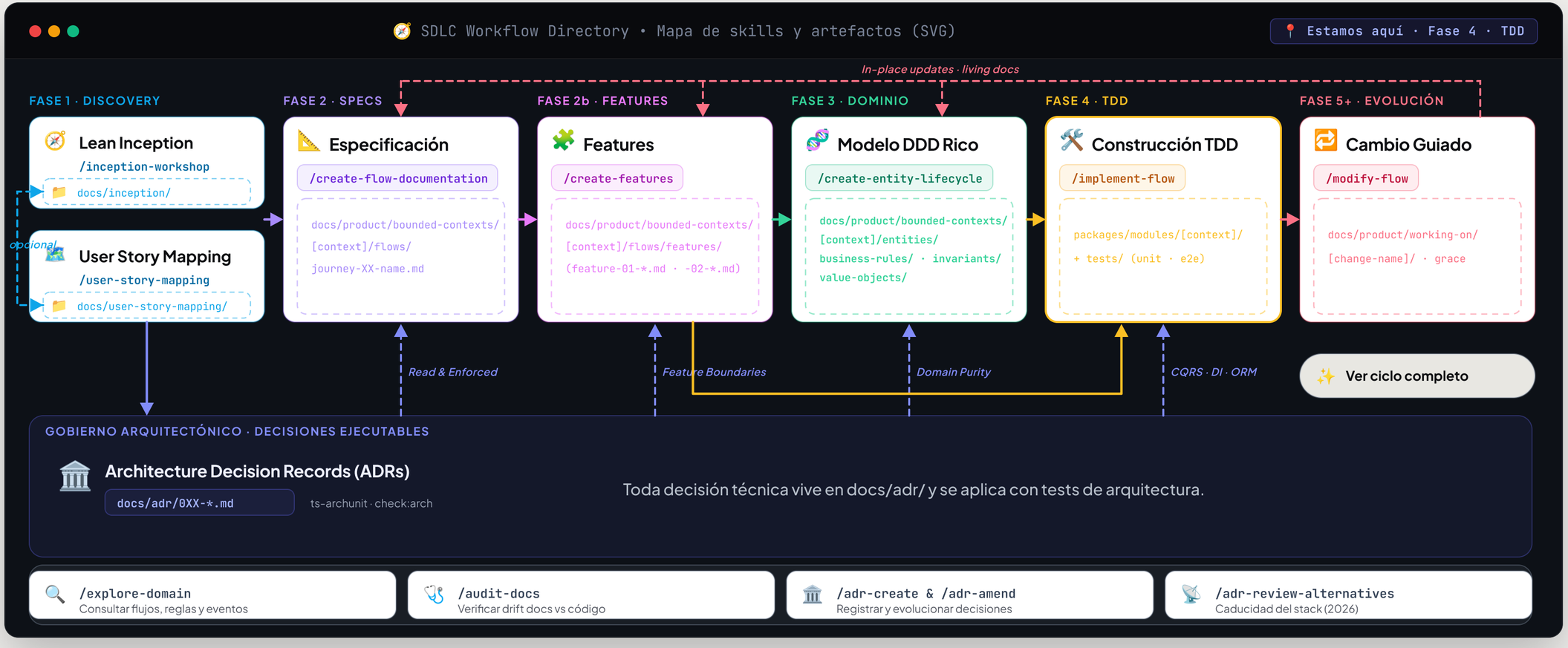
🏛️ Ley viva del repositorio

💡 **Principio Artesano:** Ninguna decisión de arquitectura se toma en un prompt efímero. Cada ADR es una ley verificada que convierte la ambigüedad en determinismo reproducible para agentes e ingenieros.

ESTACIÓN DEL MAPA • ENTRANDO EN FASE 4

## 📍 ESTAMOS AQUÍ • FASE 4: CONSTRUCCIÓN TDD

Del mapa al código: cada feature (2b) es un ciclo TDD completo — y cuando algo cambie, la Fase 5 (/modify-flow) te devuelve al mapa.



← Estación: Mapa · Fase 4

FASES 4 Y 5+ • CONSTRUCCIÓN, CAMBIO Y AUDITORÍA

# Evolución sin Drift: TDD Híbrido y Living Docs

Escribir código capa por capa guiado por tests que fallan y sincronizar la documentación en el mismo commit.

## 🧪 TDD Híbrido Capa por Capa

Fase 4 • /implement-flow

| | |
|---|---|
| 🔴 1. E2E Test (Falla) | Goal Post |
| 🔴➔🟢 2. Dominio Puro | domain/ |
| 🔴➔🟢 3. Casos de Uso | application/ |
| 🔴➔🟢 4. Infraestructura & Eventos | infrastructure/ |
| 🚀🟢 5. API & Controllers | api/ |

## 🔄 Gestión del Cambio

Fase 5+ • /modify-flow

1. Leer docs y código real
2. Redactar `proposal.md`
3. Plan por fases DDD
4. Actualización in-place

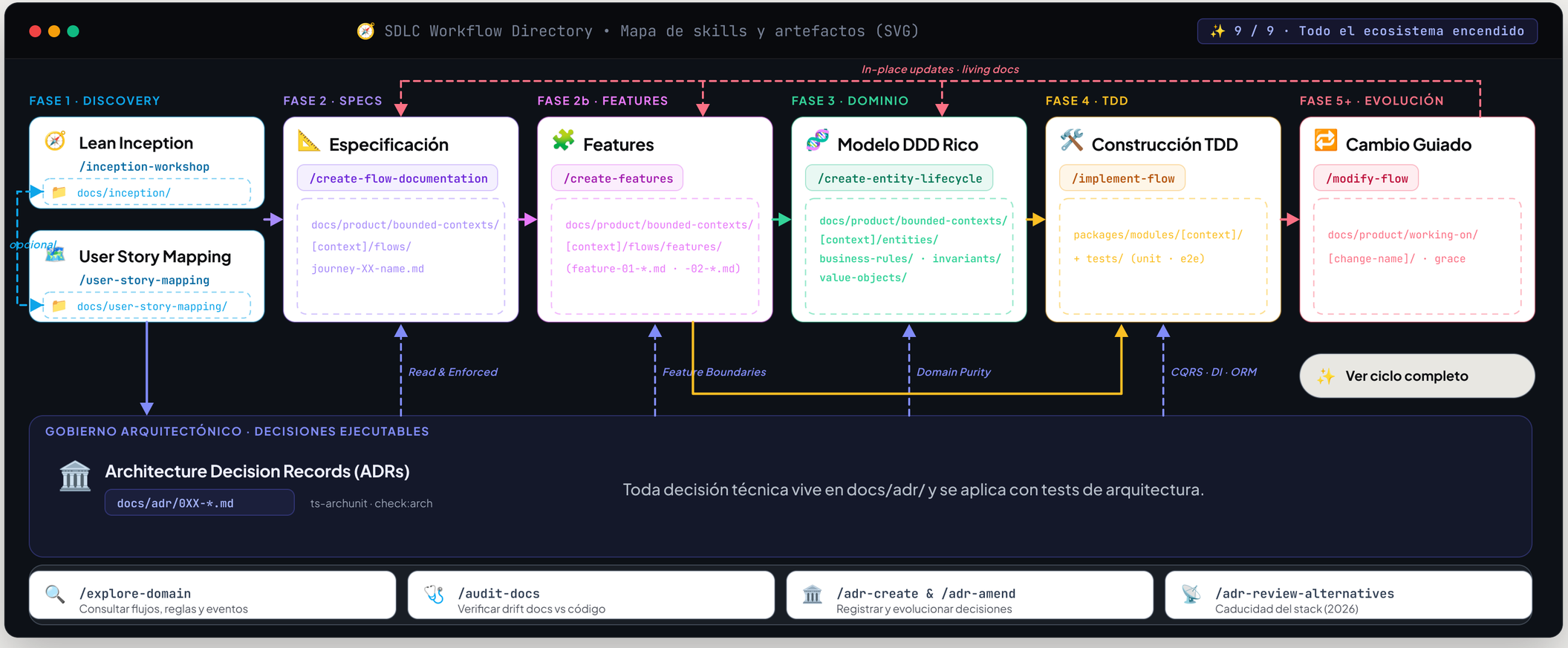
## 🛡️ Cero Drift de Documentación

/AUDIT-DOCS

✅ Confirmed ⚠️ Stale ❌ Missing ➕ Undocumented

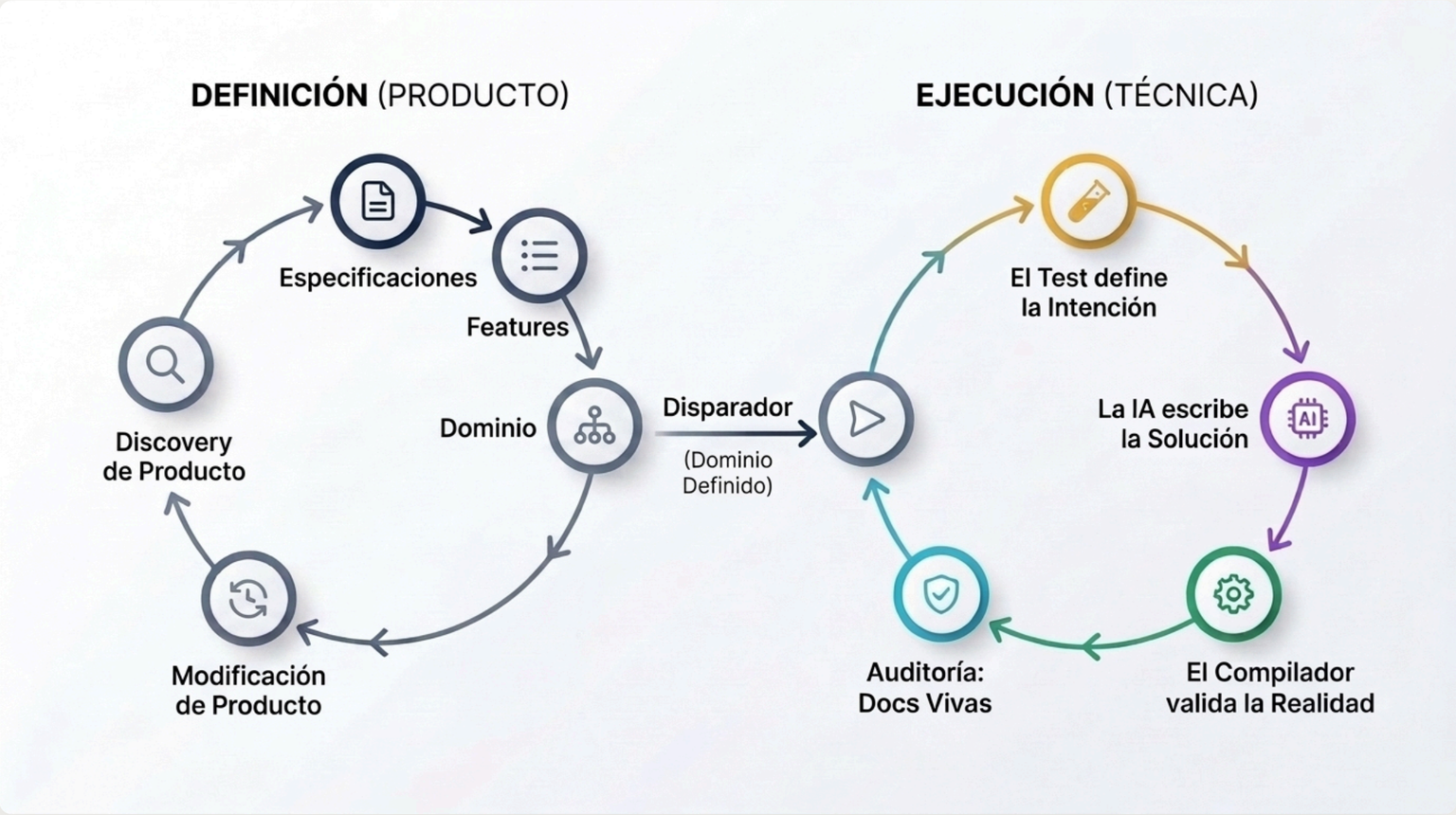
## ✨ CICLO COMPLETO • TODO EL ECOSISTEMA ENCENDIDO

Nueve pasos encendidos a la vez: del /inception-workshop al /modify-flow, con el gobierno ADR y las operaciones continuas — así sube el ecosistema entero.



FASES 4 Y 5+ • SÍNTESIS DEL CICLO

# Ciclos Conectados: la documentación nunca envejece



“La verdadera maestría artesanal consiste en entender el producto y controlar las herramientas que moldean nuestro código.”

— ARTESANÍA DE SOFTWARE EN LA ERA DE LA IA

LinkedIn

SessioFlow

SDLC Skills

La charla, en web